# Theory of Algorithms – Brute Force

Patrick Marais
Department of Computer Science
University of Cape Town
patrick@cs.uct.ac.za





# Introduction

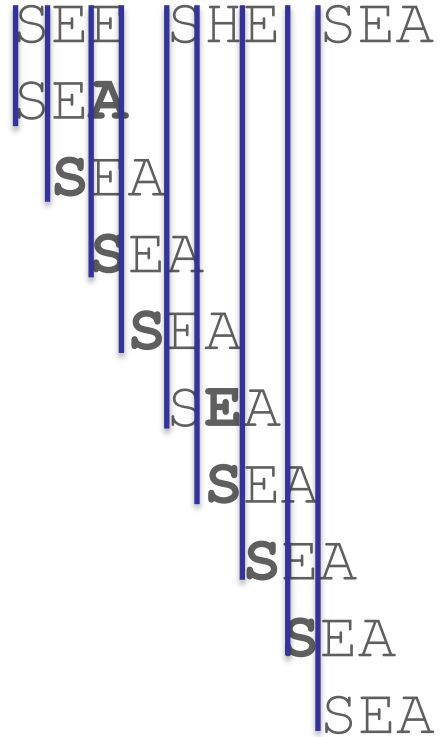
- Brute Force – A straightforward approach directly based on the problem statement
  - Implies lack of sophistication and thought
  - Exhaustive search is a special case
- However,
  - Often the quickest and easiest to implement
  - Sometimes the best solution too!
  - More importantly, often it's good enough for occasional use or small problem sizes
- Example:
  - exponentiation $a^n = a * a * a * \ldots * a$ (n times) and linear search
  - Bubble sort and insertion sort: $\Theta(n^2)$ for n input items

# String Matching

- Write a brute force solution to this problem:
  - Given a text, T, and a pattern string, P, find the index of the first occurrence in the text of P
  - If not found, return -1
- What's the worst-case efficiency class?
- Is the worst case likely?
- Think of a text and string where worst case occurs

# String Matching Example

- Brute force match “SEA” in this text



# String Matching Algorithm (High Level)

1. Align pattern at beginning of text
2. Moving left to right, compare each character of pattern to the corresponding character in text UNTIL
   - All characters are found to match (successful search); or
   - A mismatch is detected
3. WHILE pattern is not found and the text is not yet exhausted, realign pattern one position to the right and repeat step 2.

- *n* characters in Text (T) and *m* characters in Pattern (P)
- Last viable position is at n-m

# String Matching Algorithm (Low Level)

```
BruteForceStringMatch ( T[0..n-1], P[0..m-1] )
    // T is the text; P is the pattern we're searching
    // for in the text

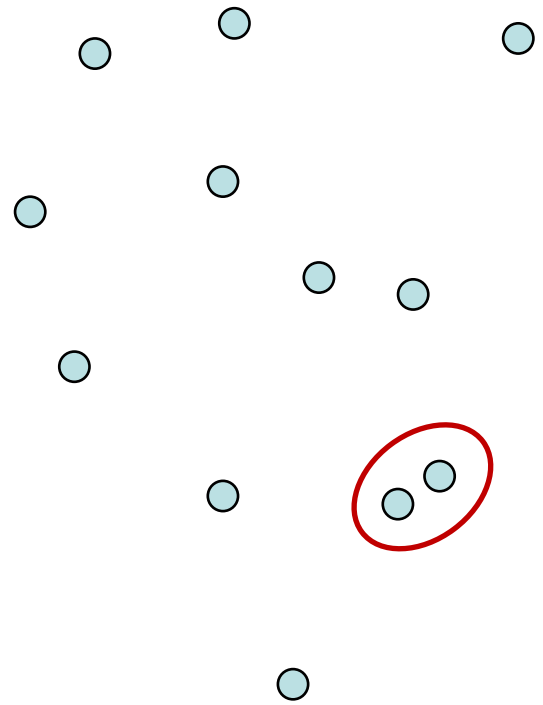
    for k ← 0 to n – m do // for each char in T
        j ← 0
        while j < m and P[j] = T[k+j] do // for each char in P
            j ← j + 1
        if j = m return k
    return -1
```

# Worst Case

- Worst case: the search string matches on every character except the last, for every iteration of the outer loop
  - E.g., Text = "aaaaaaaaaaaaaaaaaaaaaaaa...." (n chars)
  - Search string = "aaab" (m chars)
  - = m(n-m+1) character comparisons
  - = Θ(mn) for m much smaller than n (which is what happens in practice)
- Worst case very unlikely with natural language!
- Average case on natural language? – Θ(n)

# Brute Force Closest Pair

- Problem:
  - Find the two points that are closest together in a set of *n* 2D points $P_1 = (x_1,y_1)$, …, $P_n = (x_n, y_n)$
  - Reminiscent of brute-force algorithm for "is this a set"



# Closest Pair Algorithm

```
dmin ← ∞
     for i ← 1 to n-1 do
          for j ← i+1 to n do
               d ← sqrt((x_i - x_j)^2 + (y_i - y_j)^2)
               if d < dmin
                    dmin = d
                    index1 = i
                    index2 = j
return index1, index2
```

- Efficiency: $\Theta(n^2)$

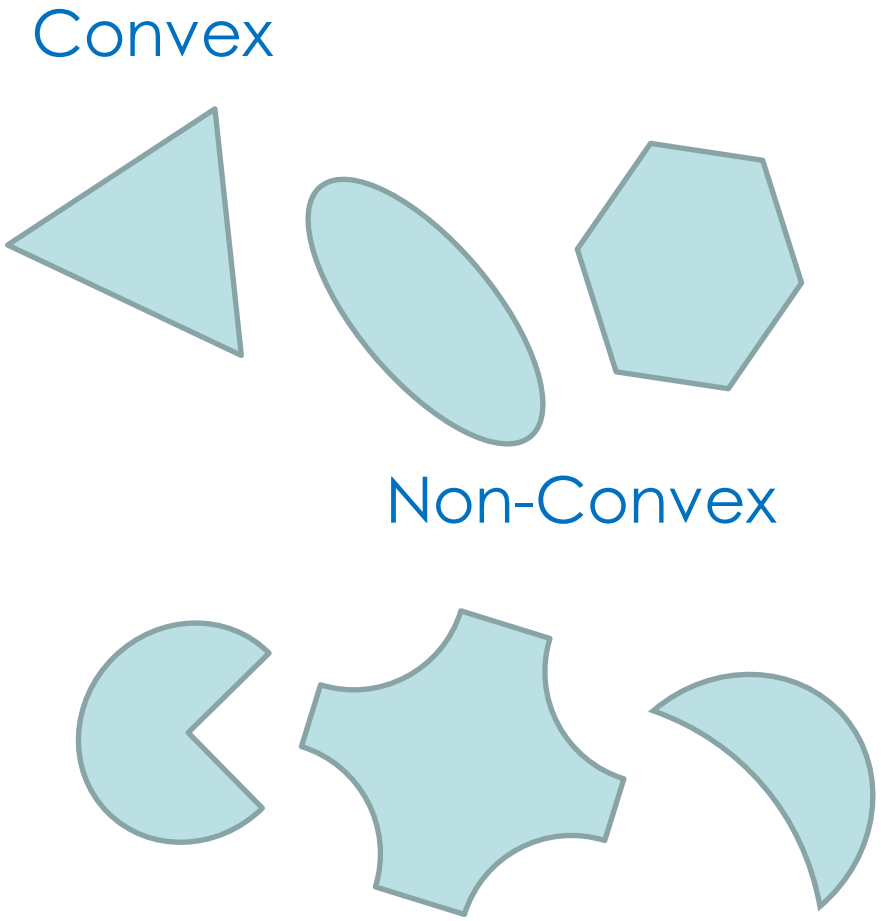
$$C(n) = \sum_{i=1}^{n-1} \sum_{j=i+1}^{n} 2 = 2 \sum_{i=1}^{n-1} (n-i)$$

$$= 2[(n-1) + (n-2) + \cdots + 1] = (n-1)n \in \Theta(n^2).$$

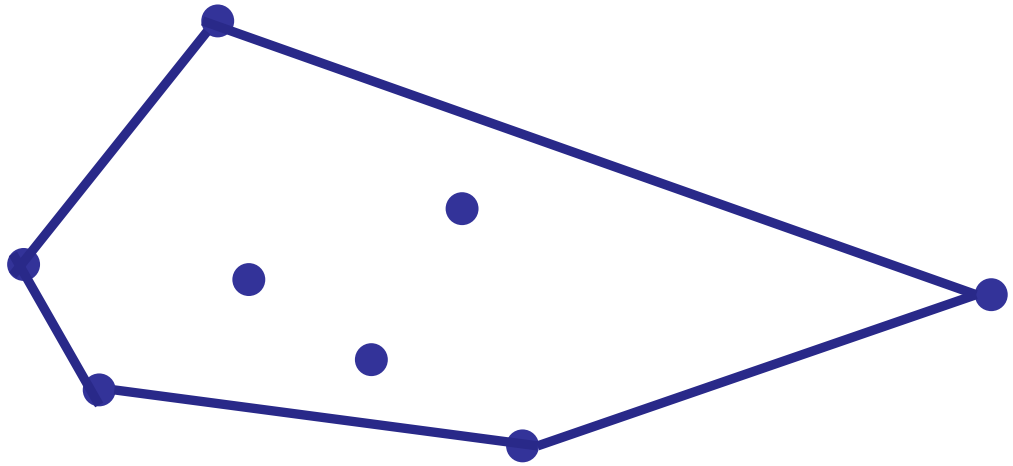
- Sqrt evaluation is expensive in practice but can be avoided

# Brute Force Convex Hull

- Problem:
  - Find the convex hull enclosing *n* 2D points
  - Convex Hull: If S is a set of points then the Convex Hull of S is the smallest convex set containing S
  - Convex Set: A set of points in the plane is convex if for any two points P and Q, the line segment joining P and Q belongs to the set



# Convex Hull Algorithm



- Algorithm:
  - For each pair of points $p_1$ and $p_2$
  - Determine whether all other points lie to the same side of the straight line through $p_1$ and $p_2$
  - They then form part of the convex hull
- Efficiency: $\Theta(n^3)$

# Pros and Cons of Brute Force

- Strengths:
  - Wide applicability
  - Simplicity
  - Yields reasonable algorithms for some important problems and standard algorithms for simple computational tasks
  - A good yardstick for better algorithms
  - Sometimes doing better is not worth the bother
- Weaknesses:
  - Rarely produces efficient algorithms
  - Some brute force algorithms are infeasibly slow

# Theory of Algorithms – Exhaustive Search





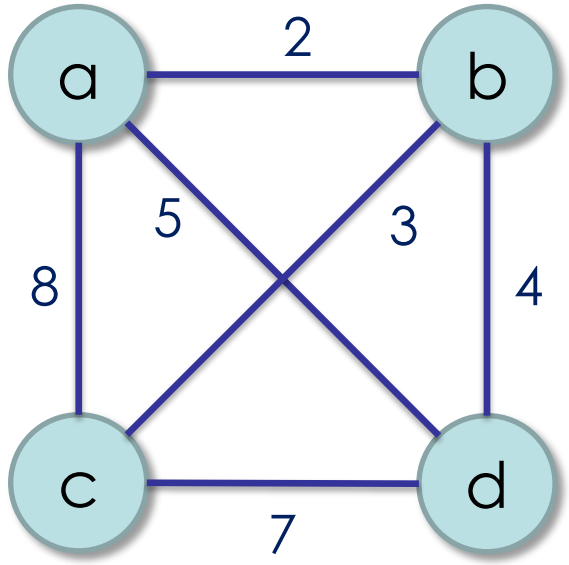
# Introduction

- Definition:
  - A brute force solution to the search for an element with a special property
  - Usually among combinatorial objects such a permutations or subsets
  - suggests generating each and every element of the problem's domain
- Method:
  1. Construct a way of listing all potential solutions to the problem in a systematic manner
     - All solutions are eventually listed
     - No solution is repeated
  2. Evaluate solutions one by one (disqualifying infeasible ones) keeping track of the best one found so far
  3. When search ends, announce the winner

# Travelling Salesman Problem (TSP)

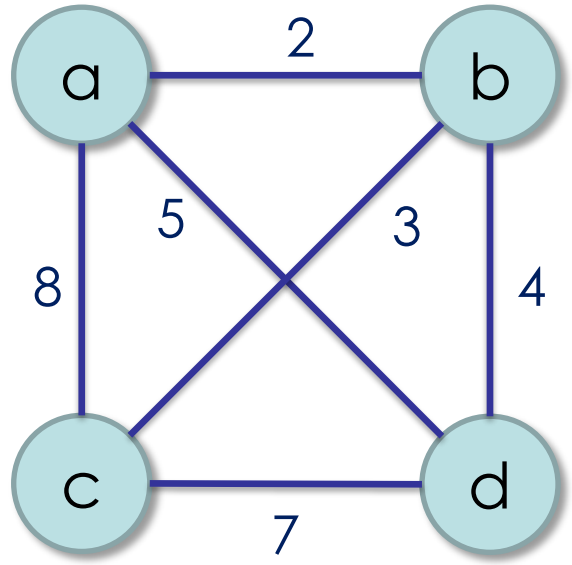
- Problem:
  - Given n cities with known distances between each pair
  - Find the shortest tour that passes through all the cities exactly once before returning to the starting city
- Alternatively:
  - Find shortest Hamiltonian Circuit in a weighted connected graph

Example



# TSP by Exhaustive Search

| Tour | Cost |
|---|---|
| a→b→c→d→a | 2 + 3 + 7 + 5 = 17 |
| a→b→d→c→a | 2 + 4 + 7 + 8 = 21 |
| a→c→b→d→a | 8 + 3 + 4 + 5 = 20 |
| a→c→d→b→a | 8 + 7 + 4 + 2 = 21 |
| a→d→b→c→a | 5 + 4 + 3 + 8 = 20 |
| a→d→c→b→a | 5 + 7 + 3 + 2 = 17 |




- All permutations of routes
- Improvements:
  - Start and end at one particular city
  - Remove tours that differ only in direction
- Efficiency: $(n-1)!/2 = \Theta((n-1)!)$

# Knapsack Problem

- Problem:
  - Given n items, with
  - weights: $w_1, w_2, \ldots, w_n$
  - values: $v_1, v_2, \ldots, v_n$
  - a knapsack of capacity W
  - Find the most valuable subset of the items that fit into the knapsack

Example: (W=16)

| Item | Weight | Value |
|---|---|---|
| 1 | 2kg | R200 |
| 2 | 5kg | R300 |
| 3 | 10kg | R500 |
| 4 | 5kg | R100 |

# Knapsack by Exhaustive Search

| Subset | Total Wght | Total Value |
|---|---|---|
| {1} | 2kg | R200 |
| {2} | 5kg | R300 |
| {3} | 10kg | R500 |
| {4} | 5kg | R100 |
| {1,2} | 7kg | R500 |
| {1,3} | 12kg | R700 |
| {1,4} | 7kg | R300 |
| {2,3} | 15kg | R800 |

| Subset | Total Wght | Total Value |
|---|---|---|
| {2,4} | 10kg | R400 |
| {3,4} | 15kg | R600 |
| {1,2,3} | 17kg | n/a |
| {1,2,4} | 12kg | R600 |
| {1,3,4} | 17kg | n/a |
| {2,3,4} | 20kg | n/a |
| {1,2,3,4} | 22kg | n/a |

- Enumerate all subsets, calculate total weight and value, choose the best
- Efficiency: $\Omega(2^n)$

# Assignment Problem

- Problem:
  - *n* people and *n* jobs to be done
  - Each person is assigned to do exactly one job
  - Each job is assigned to exactly one person
  - The cost of person *i* doing job *j* is *C[i,j]*
  - Find a job assignment with the minimum cost

Example: n = 4

| | JOB1 | JOB2 | JOB3 | JOB4 |
|---|---|---|---|---|
| **PSN1** | 9 | 2 | 7 | 8 |
| **PSN2** | 6 | 4 | 3 | 7 |
| **PSN3** | 5 | 8 | 1 | 8 |
| **PSN4** | 7 | 6 | 9 | 4 |

# Assignment by Exhaustive Search

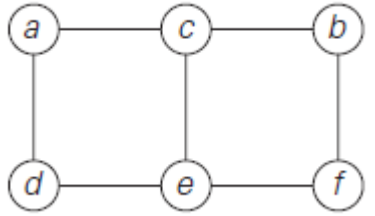
- Select one element in each row in different columns
  - e.g., {1, 4, 3, 2} means PSN 1 gets JOB1, PSN2 gets JOB4, …
- Solution:
  - Generate all permutations of n positive integers
  - Computer the total cost for that assignment
  - Retain the cheapest
  - Poor efficiency: Θ(n!) – we can do better

| | JOB1 | JOB2 | JOB3 | JOB4 |
|---|---|---|---|---|
| **PSN1** | 9 | 2 | 7 | 8 |
| **PSN2** | 6 | 4 | 3 | 7 |
| **PSN3** | 5 | 8 | 1 | 8 |
| **PSN4** | 7 | 6 | 9 | 4 |

| | JOB1 | JOB2 | JOB3 | JOB4 | COST |
|---|---|---|---|---|---|
| **{1,2,3,4}** | 9 | 4 | 1 | 4 | 18 |
| **{1,2,4,3}** | 9 | 4 | 8 | 9 | 28 |
| **{1,3,2,4}** | 9 | 3 | 8 | 4 | 24 |
| … | … | … | … | … | … |

# Exhaustive Search – graphs

- Exhaustive search in graph traversal – visit all graph vertices
- Two standard traversals
  - Breadth first search (BFS) – uses stack
  - Depth first search (DFS) – uses queue
- Traversal time for both is
  - $\Theta(|V|^2)$ for adjacency matrix rep
  - $\Theta(|V| + |E|)$ for adjacency list rep
- Where $|V|$ is number of vertices, $|E|$ is number of edges




| | | | | | |
|---|---|---|---|---|---|
| a | → c | → d | | | |
| b | → c | → f | | | |
| c | → a | → b | → e | | |
| d | → a | → e | | | |
| e | → c | → d | → f | | |
| f | → b | → e | | | |

| | a | b | c | d | e | f |
|---|---|---|---|---|---|---|
| a | 0 | 0 | 1 | 1 | 0 | 0 |
| b | 0 | 0 | 1 | 0 | 0 | 1 |
| c | 1 | 1 | 0 | 0 | 1 | 0 |
| d | 1 | 0 | 0 | 0 | 1 | 0 |
| e | 0 | 0 | 1 | 1 | 0 | 1 |
| f | 0 | 1 | 0 | 0 | 1 | 0 |

# Comments on Exhaustive Search

- Exhaustive search algorithms run in a realistic amount of time only on very small instances
- In many cases there are much better alternatives!
- In some cases exhaustive search (or variation) is the only known solution
- and parallel solutions can speed it up!